1 - Analysis of Requirements for Testing Strategies

> snnTorch is a Python package for performing gradient-based learning with spiking neural networks.
> It extends the capabilities of PyTorch, taking advantage of its GPU accelerated tensor computation and applying it to networks of spiking neurons.
> Pre-designed spiking neuron models are seamlessly integrated within the PyTorch framework and can be treated as recurrent activation units.

For snnTorch, our collection of requirements should combine the following:

1. mathematical correctness,
2. reliable training,
3. PyTorch integration,
4. usability,
5. resource use,
6. secure dataset handling.

Stakeholders and evidence:

1. Researcher needs correct neuron dynamics
2. Beginner needs understandable instructions
3. Maintainer needs failures that are easily reproducible and diagnosable.

Verification: does snnTorch confirm to a precise requirement?
Validation: does that requirement address the stakeholder's actual need?

Unit means an individual function or neuron.
Integration means collaborating components, including PyTorch.
System means installed package used in complete workflow.
Acceptance testing adds assessment by representative users.

SNN Researchers + Computational Neuroscientists

- Need confidence that experimental results reflect the intended models and algorithms.
- The relevant implementations include Leaky neurons, surrogate gradients, and spike encoding.
- Requirement: neurons follow their own mathematical models, including decay, spike thresholds, reset methods, reset timing, etc.
  - Type = Functional correctness
  - Level = Unit
  - Test: we should compare membrane values and spikes over time with an independent mathematical reference. this should include each reset method, scalar

and vector parameters, and values below, at, and above the threshold. spikes should match exactly, and membrane values should match within some agreed numerical tolerance.
    - Strengths: gives clear expected results and can detect numerical errors or incorrect timing
    - Limitations: the reference model could also be wrong, so we need an expert to check the specification
- Requirement: surrogate gradients implement their specified forward spike rule and backward gradient rule properly.
    - Type = functional correctness
    - Level = unit
    - Test: we should compare outputs and gradients with independently calculated formulae across inputs and parameter values. should include zero, positive, negative inputs, batches, and a whole range of parameter values.
        - Strengths: directly checks intended maths
        - Limitations: finite differences of the hard spike function are unsuitable because its derivative is intentionally different to the surrogate rule
- Requirement: spike encoders should represent inputs correctly. rate encoding must produce the intended probabilities, latency encoding must produce the intended spike times, and one-hot encoding must preserve class labels.
    - Type = functional correctness (including for stochastic behaviours)
    - Level = unit
    - Test: we test this by checking rate probabilities of 0 and 1 exactly, and for intermediate probabilities, we use a binomial acceptance interval chosen before testing. we compare latency outputs with calculated spike times, and test one-hot encoding with repeated and shuffled labels.
        - Strengths: use exact checks where possible and statistical checks for random outputs
        - Limitations: statistical tests can occasionally reject a correct encoder, so we should set sample sizes and significance levels in advance. each encoder also needs its own detailed requirements.

Application Developers and Deployment Engineers

- Need dependable operation and predictable costs.
- State isolation is particularly relevant because neuron instances are registered globally, and reset utilities use class-level operations.
- Requirement: independent models should not unintentionally affect each other, and starting a new independent sequence through the documented reset procedure should reproduce behaviour as if it were a fresh model.
    - Type = State isolation and reliability
    - Level = Integration

    - Test: we compare sequence B after processing A and resetting against B on a fresh identical model, then interleave the two models - reset one and verify the other's state remains unchanged. we repeat across 1,000 declared sequence/model combinations.
        - Strengths: this targets state leakage and dependence on execution order
        - Limitations: these repetitions do not estimate real-world reliability unless the workloads are designed to represent actual usages.

Students, Educators, First-Time Users

- Need an approachable workflow and accurate explanations
- The repository's README and examples provide the starting material.
- Requirement: a user with basic PyTorch knowledge can install the package and run a minimal SNN example using the supplied instructions given. It should be that at least 4 of 5 such users can complete this within 30 minutes without an instructor intervening.
    - Type = Measurable usability
    - Level = System
    - Test: We test this by conducting task-based sessions in clean environments, recording completion, time, errors, and requests for help. We should also keep note of whenever things are failing because of installation errors instead of issues with conceptual understanding.
        - Strengths: Directly validates users' experiences
        - Limitations: We cannot do this for large population size in Edinburgh, but so five participants is not enough for a population-wide usability estimate.

Dataset Users and Administrators Running Data Pipelines

- Need correct event conversion, dependable access, protection of local files
- The fork includes event slicing and dataset download/extraction.
- Requirement: We must ensure when getting a dataset that dataset extraction cannot be written outside its destination, and secure downloads are actually secure.
    - Type = Functional security
    - Level = Unit/Integration. Also system if we test full download + extraction workload
    - Test: In a disposable file system, we test archives containing parent-directory paths, absolute paths, and escaping links. We then verify rejection and unchanged external files that are placed somewhere so a test can detect unintended changes. We also simulate HTTPS failure and inspect fallback behaviours.
        - Strengths: targets clear security properties that have observable outcomes
        - Limitations: these tests cover only specific attacks, so not exhaustive. we should also inspect extraction code to check how it handles paths and links that could escape the destination folder, and use an extraction method with

documented security protections, rather than relying on custom checks that may miss cases.

Chapters 2 + 3

Principle	Application to snnTorch
Sensitivity	Make invalid configurations fail consistently. Expose state leakage through controlled interleavings.
Redundancy	Express intended equations and contracts independently, then check implementation consistency.
Restriction	Declare supported environments and input domains. Use explicit state and fixed randomness where appropriate.
Partition	Divide testing by level and by meaningful cases: reset modes, threshold boundaries, devices, and valid/invalid inputs.
Visibility	Trace each requirement ID to tests and results. Retain diagnostic membrane traces, gradients, and benchmark conditions.
Feedback	Turn discovered defects into regression tests and revise partitions or review checklists when existing checks miss them.